Decentralized Federated Learning Over Imperfect Channel

Aditya Kumar Sahu, Rishav Kumar Manjhi, D. Yaswanth Reddy
Department of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, 632014, Tamil Nadu, India
Corresponding author: aditya.kumarsahu2024@vitstudent.ac.in

Abstract

Decentralized Learning Exchanges are peer-to-peer models that exchange information among peers by sending model parameters to each other's neighboring client but do not have an aggregation center. This paper identifies the conditions at which the sharing of information (communication) will enhance learning with regard to the cost of the sharing in an environment where channels can be degraded. A PyTorch-based simulator was developed to compare learning in three different environments: a local-only environment, a standard gossip environment, and an event-driven learning environment. The simulation environment used both iid and non-iid (two-class) label partitioning as well as Dirichlet. For the MNIST experiment, we ran five separate simulations seeded independently to test stability. Each simulation ran for ten iterations of gossip, and each iteration of gossip included one epoch of learning. The CNN architecture had 206,922 trainable parameters. The iid condition resulted in a mean accuracy of approximately 98.03 percent with a standard deviation of approximately 0.08 percent for the local-only case. In contrast, the non-iid (two-class) condition resulted in a mean accuracy of approximately 19.95 percent with a standard deviation of approximately 0.003 percent. Furthermore, ring-gossip achieved a significantly higher accuracy rate of approximately 49.27 percent ± 3.13 percent for the same non-iid (two-class) condition. Therefore, these controlled studies demonstrate that although achieving very high iid MNIST accuracy may provide some measure of confidence regarding robustness of the communications channels, it does not ensure robustness. Additional experiments were conducted to explore additional factors including the effects of delay and independent/burst packet loss. Network size was also explored as a factor. Finally, CIFAR-10 was added as another dataset for comparison. To provide a quantitative measurement of traffic reduction from the communication protocol, all directed message counts and payload byte counts were collected during execution. Spectral analysis was performed to confirm that matrix averaging occurs separately from local-SGD.
The results presented herein describe the specific topologies and configurations tested as opposed to providing a general topology ordering or convergence theorem for impaired training.
Keywords: decentralized learning; gossip; non-iid; event triggered communication; packet loss; delay; spectral gap; reproducible.

1 Introduction

1.1 Background

Federated Learning allows Clients to use multiple Datasets while sharing Model Information rather than transferring Raw Data from each Client to a Central Location [1] . Within Decentralized Systems (such as IoT), adjacent Devices will share updates among one another. Due to this Design Architecture, there are several Edge and IoT Deployments in which it would be either Costly or Not Possible for a Central Service to Aggregate Data. Data Locality is a Property of how data is handled; a Formal Privacy Guarantee will require an Additional Mechanism.
The Network Structure determines who can communicate their Model Updates to whom, while the Unreliable Delivery of these Updates determines which communications affect the overall Model. Therefore, the overall Behavior of the Model, when combined with the Distribution of the Local Dataset(s) and the total amount of Computation performed by the Client must be Evaluated.

1.Introduction Continued

1.2State-of-the-Art

Decentralized versions of Stochastic Gradient Descent (SGD) and Decentralized Federated Averaging have created an environment in which a server may not be needed during the process of training using local optimization and neighbor exchange [2], [3]. Furthermore, consensus based IoT learning and peer-to-peer graph learning show that there are ways to organize cooperative processes without a central model aggregator [4], [5]. Thus, these works motivated our development but the update mechanisms and convergence conditions are specific to the respective methods.

Unreliability in communication has been extensively studied. For example, soft-decentralized SGD has both utilized partially received information and developed reliability aware mixing; and D2D learning studies have specifically analyzed unreliably exchanged wireless messages. In addition, Li et al. have evaluated the aggregation bias and the amount of local aggregation necessary for perfect communication under various levels of channel impairment [6] [7] [8]. Additionally, communication efficient methods utilize techniques such as compression, sparse exchange and adaptively communicating instead of simply suppressing all communication [9] [10].

1.3The Research-Gap

Here we investigate how much improvement one obtains by implementing a distributed gossip algorithm versus locally training a model alone, under varying degrees of data heterogeneity, suppressed communication and channel degradation. Here, we investigate what relationship exists between this degree of improvement and actual payload traffic as well as model disagreement. We will examine at what point the ideal spectral predictions no longer provide a full description of the entire learning procedure. This investigation gap was formally identified in section2.4.

1.4 Objectives

1. Determine the accuracy increase in communication versus matched local-only control across IID and non-IID data distributions.
2. Investigate the effects of independent erasure, burst loss and delay due to finite rounds of delivery on accuracy using explicit message-tracking.
3. Characterize the accuracy vs. traffic trade-off of using fixed event thresholding for either directed-messages or payload-bytes.
4. Investigate Ring, Grid, and Random graph topologies and determine if conclusions obtained in smaller network environments continue to hold true as the number of clients increases.
5. Validate ideal spectral averaging computations independently from empirical neural network training.

1.5 Contributions

M1 develops a local-SGD/neighbor-averaging simulator with reproducible simulation results, version-aware delayed delivery and measured aggregation-induced mean drift. M2 extends M1 with sender side event generation while maintaining the same receiver interface as M1. M3 extends the previous models to calculate signed spectra, SLEM and a squared-disagreement bound for ideal fixed average computing. Empirical contributions consist of 155 runs across five different seeds utilizing local-only controls, heterogeneous data, channel comparisons, larger networks and a limited CIFAR-10 expansion. Our work investigates the characterization of this implementation as opposed to developing a new convergence theory.

2 Literature Review

2.1 Decentralized Federated Learning and Gossip Based Optimization

The work by Xiao and Boyd on distributed average computations has provided a theoretical foundation for exchanging information across graphs. Boyd et al. also investigated randomized gossip as related to spectral properties of matrices [11], [12]. While these works provide theoretical insights into specific averaging operators that can be applied to computing the average of a graph (or network), they do not consider the specifics of the local optimization mechanisms in use nor the communication mechanisms used between optimizations. Lian et al. analyzed the decentralized parallel version of stochastic gradient descent (SGD) as well as Sun et al.'s analysis of decentralized federated averaging with momentum and a quantized version [2], [3]. While both models link computation at each node to communication from neighboring nodes, it is unclear whether an empirical study based upon implementing an optimizer with retained momentum for each node and normalized to each receiving node will inherit all of the theoretical results of either prior study. Therefore, our experiment tracked the actual update policy used during training, so that we could evaluate the performance of the learner empirically. Hegedűs et al. have compared gossip learning with federated learning in terms of communication cost and differing data distributions. Savazzi et al. developed cooperative strategies for IoT devices through a consensus-based approach, while Lalitha et al. have proposed a graph-based peer-to-peer learning algorithm using Bayesian-like belief updates [13], [4], [5]. Thus there exist many possible ways to organize collaboration among learners; however, the various studies are not equivalent implementations of some single averaging algorithm.
In addition to providing insight into collaborative organization, Gao et al. have discussed the relationship between decentralized optimization and federated learning, and Liu et al. have examined how multiple local iterations and communication steps trade off computational cost versus achieving agreement between learners [14], [15]. Given this tradeoff, we chose to specify separate budgets for training and communication. For example, we fixed ten rounds and one local epoch for our primary comparison, and a separate IID control repeated the original twenty-round and two-local-epoch budget.

Implication for this study

In order to make an accurate comparison, we need a no-communication control, a clear local update policy, and a clearly defined measure for how the clients' weights are mixed. Therefore, we will be comparing individual client models using the same partition as well as equalized starting values, maintaining optimizer momentum within each client's model, and making it clear that our metric will assess the accuracy of predictions made by each client separately from their degree of similarity with regard to parameters in order to accomplish this.

2.2 Communication Efficiency and Learning over Imperfect Channels

Communication may be minimized through transmitting fewer and less voluminous updates, selecting alternative peers for each update, or altogether eliminating opportunities for transmissions to take place. Koloskova et al. discuss compressed decentralized optimization. In gossipFL both model sparsity is combined with adaptive communication and peer selection [9][10] and in our M2 trigger we completely eliminate an entire float32 model broadcast. Compressed distortion and message erasure are two different mechanisms therefore findings related to one mechanism do not serve as direct proof of the other.

In addition to how frequent updates are sent, wireless learning also depends upon how those updates are transmitted. Xing et al. examined digital and analog decentralized learning in D2D networks (including channel impacts), assuming certain conditions were met. Chen et al. reviewed distributed learning across a broader scope of wireless networks [16][17]. This particular simulator operates at the application-message level: it examines erasures and stale arrivals but does not simulate the generation of an analog waveform or transmit via a full physical layer channel.

The most similar studies concerning unreliable links include Ye et al., Jiang et al., and Li et al. [6][7][8]. Ye et al. optimized mixing using reliability data associated with individual links. Li et al. studied imperfect aggregation and its relationship to model bias and aggregation frequency. Our receiver-normalized baseline did not replicate these algorithms. Instead, it serves as a controlled reference point whose average rate of drift and finite-budget behavior can be measured directly.

Studies on centralized FL and cellular FL provide additional viewpoints. Yang et al. studied scheduling policies. Salehi and Hossain examined unreliable cellular networks which operate under constrained resources. Li et al. studied fairness across unreliable IoT links [18][19][20]. While the topics motivating us to report the actual amount of communication and client-level performance are relevant to our work, centralized scheduling and optimizing fairness are outside the protocol implemented here.

Over-the-air and noisy-channel studies require additional assumptions. Xu et al. investigated learning-rate optimization for over-the-air computation. Amiri et al. analyzed a noisy downlink. And Wei and Shen studied learning across noisy channels [21][22][23]. The continuous error caused by noisy channels are clearly unrelated to individually erasing all content from entire model messages. Therefore we used such studies to define the limits of the wireless setting as opposed to validating our erasure models against their analyses.

2.3 Network Topology Consensus and Convergence

Nedić et al. outline the influence of graph structure and assumed communication on the performance of a decentralized optimizer in their paper [24] . This provides for the possibility to use an ideal mixing matrix as a design descriptor. However, it is not possible to make a prediction based upon a universal application of the ideal mixing matrix's spectral gap to the eventual neural-network accuracy. A variety of factors such as local gradients (including those produced due to heterogeneity), directed missing messages, and stale parameters will alter the relationship between an ideal graph property and measured disagreements in models.

Martinez-Beltran et al., along with Yuan et al. have surveyed Distributed Federated Learning (DFL) using the categories of Architectures, Communication Mechanisms, Topology, System Design and Open Challenges respectively [25],[26]. Chung et al. provide an additional review of Non-IID DFL, which includes Architecture, Topology, Optimization, Security and Evaluation Methods [27]. All of these references provide justification for performing evaluations which clearly state the data distributions and system assumptions used as opposed to making assessments about the accuracy of IID.

Additional research has identified other limitations. Chen et al. developed a differentially private wireless DFL scheme. Imteaj et al. provided a resource constrained IoT learning survey. Nguyen et al. studied self organizing distributed learning. Finally, Chen et al. investigated mobility aware vehicular DFL [28],[29],[30],[31]. We did not include privacy protection, autonomous network organization and vehicle mobility in our static-graph simulator. These represent extensions to what could be determined if we keep sample data localized.

Research strand	**Representative references**	**Role in the present study**
Optimization and averaging	[11], [12], [2], [24], [3]	Defines ideal mixing and local-update foundations; assumptions are stated explicitly.
Unreliable decentralized exchange	[16], [6], [7], [8]	Motivates loss, stale-arrival, and average-drift measurements.
Communication efficiency	[9], [10], [15]	Distinguishes whole-message suppression from compression and adaptive peer selection.
Heterogeneity and system evaluation	[25], [26], [27]	Motivates explicit non-IID settings and consistent evaluation metrics.
Privacy resources and mobility	[28], [20], [29], [30], [31]	Identifies capabilities that remain outside the implemented simulator.

Table 1. Literature synthesis. The table classifies the selected studies by their main relevance; it is not a claim that each paper studies only one issue.

2.4 Research Gap and Positioning of the Present Study

The body of work in which we will be building upon has established both decentralized optimization, wireless D2D learning, communication-efficient gossip algorithms, and learning over noisy (imperfect) channels. Therefore our research gap can be identified through controlled comparison questions: What changes to the amount of communication needed (cost) and/or its value, occur due to varying levels of data heterogeneity; different graph structures; and the level of successful message delivery among receivers; all under matching training conditions; when using the specified receiver-normalized gossip algorithm?

Firstly, although achieving high accuracy on a simple IID task is indicative of good performance by a method, it is not necessarily indicative of how much each of these methods contribute to this good performance. Therefore we addressed this measurement gap by adding three types of measurements to the study: First, we added paired local-only controls to measure what portion of the collaboration was used. Second, we created exactly two-label client partitions to create data heterogeneity. Additionally, we also partitioned clients based on their respective Dirichlet distributions, with $\alpha = 0.1$.

Thirdly, simply suppressing a client's broadcast is not the same as reducing the number of neighbor messages communicated, especially considering an irregular graph where there may have been many more attempted broadcasts than actually sent messages. As such, we accounted for all actual directed attempts (messages received) and payload bytes.

Fourthly, while ideal averaging theory provides an ideal operator to average across nodes in a network, learning requires repeated application of local updates. Thus, we tested the ideal scalar bound independently and compared the accuracy of trained models at various numbers of clients without implying that a greater ideal spectral gap will provide better accuracy.

Evaluation question	**Literature basis**	**Evidence provided here**
When does communication improve learning?	Peer-to-peer learning, decentralized averaging, and non-IID DFL [5], [3], [27]	Matched local-only controls; IID, two-class, and Dirichlet comparisons; paired five-seed gains.
How do losses and stale arrivals change outcomes?	Unreliable-link DFL and imperfect aggregation [6], [7], [8]	Bernoulli and Gilbert-Elliott erasures; maximum delay 0, 1, 2, 4; arrival accounting and mean drift.
What does an event trigger actually save?	Compressed, sparse, adaptive, and budget-aware communication [9], [10], [15]	Fixed-threshold accuracy-traffic curves; broadcasts, directed messages, and bytes counted separately.
Which spectral conclusions transfer to training?	Distributed averaging and topology-dependent optimization [11], [12], [24]	Signed spectra and ideal squared-error bounds; separate N=5/20/50 learning comparisons.

Table 2. Research questions derived from the literature and the corresponding experimental evidence. These are questions evaluated by this project, not assertions that all earlier work omitted them.

The contribution is a single reproducible measurement protocol and the supporting evidence for that protocol. It includes comparisons across 31 different conditions using all five seeds. The CIFAR-10 experiment has been reduced to a comparison of local-only versus Ring training. Additionally, it also provides a study on the impact of scale by comparing Ring and Grid training on a fixed total amount of data. There are no benchmarks provided to compare our methods against implementations of Soft-DSGD, GossipFL or other methods found within the literature, therefore the results do not provide an indication as to whether they are superior than these existing algorithms.

3 Methodology

3.1 Problem Formulation

The client i has a disjoint data set D_{i} which includes a local mean cross-entropy loss function F_{i}.The objective weight values for the clients are to be equal since they have been uniformly weighted. Thus, we differ from a sample-weighted pooling of data objectives when using different Dirichlet partitioning for each client. For each seed, both the compared methods and the method being tested use the same partitioning, reference weights, hyperparameter values per client, and minibatch orderings.Possible values w is given by $F(w)=N^{-1}\sum_{i}F_{i}(w)$.

$$min\ F(w)\ =\ N^{-1}\ \Sigma_{\mathrm{i}}\ F_{\mathrm{i}}(w) \quad (1)$$

We used a single local SGD iteration per round in our core experiments. We used 2 iterations for the original-budget control. Both the learning rates were 0.01 and both had momentum at 0.9. There was no sharing of optimizer momentum. Each client's model parameter is combined; there is no combination of optimizers. All methods share the same policy regarding optimization. The core experiment had a total of 10 local epochs per client whereas the original-budget control had a total of 40.

Ring Network Topology

Ring is a ring topology that is formed by cycling through N nodes. A grid network consists of a rectangular grid of size times G, where the number of rows, $\sqrt{N}$, is the largest integer less than or equal to the square root of the number of nodes, {N}. The number of columns, G(N,0.5), is such that it contains all N nodes. A random network is constructed by sampling an Erdős-Rényi graph with N nodes and edge probability 0.5. In our experiments, graphs are fixed and undirected; however, each raw run stores the actual edges present. The experimental seeds will generate different realizations for the random networks, while the illustrative five-node spectral table will be generated based on seed 42.

$$W_{\mathrm{i}}\ =\ 1\ /\ [1\ +\ max(deg(i), deg(j))],\ (i,j)\ \in\ E \quad (2)$$

$$W_{\mathrm{ii}}\ =\ 1\ -\ \Sigma\ \neq_{\mathrm{i}}\ W_{\mathrm{i}} \quad (3)$$

In general, given an undirected graph, the original Metropolis-Hastings matrix is symmetric and doubly stochastic. Let $R_i(t)$ represent the neighbors of node i that receive their newest acceptable message within round t. Node i updates its model after receiving acceptable messages from its neighbors as follows:

$$w_{\mathrm{i}}^{+}\ =\ [W_{\mathrm{ii}}\ w_{\mathrm{i}}\ +\ \Sigma\ \in R_{\mathrm{i}}(t)\ W_{\mathrm{i}}\ w(received)]\ /\ [W_{\mathrm{ii}}\ +\ \Sigma\ \in R_{\mathrm{i}}(t)\ W_{\mathrm{i}}] \quad (4)$$

If no acceptable message arrives then node i maintains its current local model. Missing directed messages independently do not affect row-normalization but may destroy column-normalization. Because the arithmetic network mean can change during aggregation, we cannot describe this procedure as globally unbiased averaging with respect to loss.

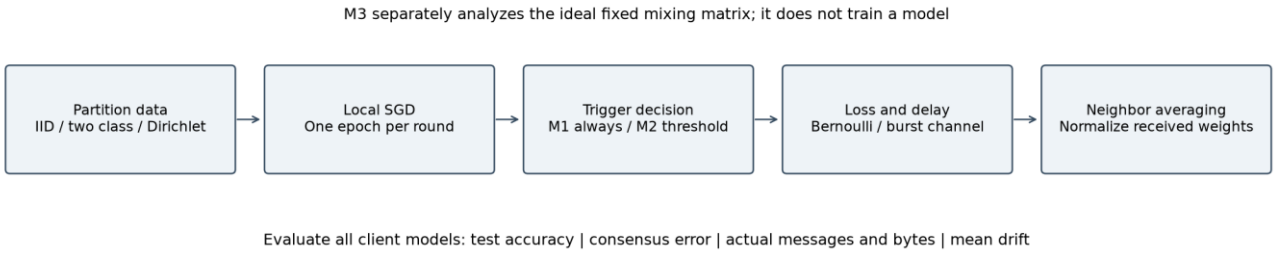


Figure 1. Common training workflow. M3 analyzes the ideal graph separately from neural-network learning.

3.1 Problem Formulation continued

Channel Model

Loss and burst model

Text rewritten to sound like a person:
In other words, the loss from impairment that occurs when two messages (one of which is "directed") are compared, is randomly erased (by an independent loss) with a .2 probability. In addition, randomness is tied to several things including: seed, sender, receiver, round and stream. Therefore, suppressing an irrelevant message does not cause the channel's output to be different for a particular link and round.

The Gilbert Elliott model has probabilities follows:
the-good-to bad transition probability is .05;
and the bad to good transition probability is .20 per round.

When in the good state there is no erasure of data; but when in the bad state all data is lost.
The stationary erasure probability = 0.05/(.05+.20) = 0.20, which corresponds to the independently-occurring-losses' specified rate.
A mean duration of the bad state is 5 rounds.
Each link begins in stationarity and passes through rounds without transmitting data.

Delay and receiver freshness

A remaining message is subject to an independently drawn uniform integer delay of zero to dm_{ax} inclusive. For maximum delays of 1, 2, and 4 versus no delay under 20% Bernoulli loss. Metadata about source rounds guarantees that the most recent arrival from a sender will be chosen; an older version can't replace a previously accepted version. Discarded delivered versions (stale) continue to utilize communications.
Pending packets scheduled after the last round remain in a pending state. Counts of attempted, dropped, received, and pending messages are all independent. The final model was not changed by any post training buffer draining. Delivered bytes are based on those received during the observation time frame and do not include those that may have been scheduled for delivery at some point in the future.

M2 Event Triggered Gossip

$$send_i(t) \Leftrightarrow \delta = 0 \; or \; ||w_i(t) - w_i(last\ sent)||_2 > \delta \quad (5)$$

A triggered client will send an update to all of its neighbors and it will also update the snapshot that it has sent most recently regardless of whether or not the channel later drops the update. The silent client is able to receive and average over time. Threshold of "zero" does imply that you would always be transmitting. And thresholds of .50 and 2.00 were chosen as a way to create static comparisons for testing purposes versus being threshold values which were optimized based on testing error.

$$message\ savings\ (\%) = 100\ [1 - actual\ attempts\ /\ (2\ |E|\ T)] \quad (6)$$

Broadcast suppression takes advantage of all N T (opportunities). A client's message saving weights are determined based upon its silence level. Percentages will be consistent across a ring but may vary with grid or random graphs. Bytes sent as payload = attempted messages *float32 model size; no network-protocol overhead included.

3.2 Data and Materials

Datasets and Client Partitions

The MNIST dataset is composed of 60,000 training and 10,000 test images for written numbers. Each input image is grayscale (i.e., black-and-white) in size 28 x 28 pixels and has been normalized to divide values by 255. This was done after subtracting the mean value (which equals 0.1307) from each pixel value and dividing it by that pixel's respective standard deviation (0.3081). Indices were allocated independently and identically distributed (IID), shuffled, and then divided. This created the two-class partition where every client had both of one digit label and that digit was assigned to both of the client's labels. Class-wise $\alpha = 0.1$ Dirichlet allocations were used to create the partitions; if there would be less than 100 samples of either class at any client when allocating them, this split would be rejected. As a result, every split represents an exact subset of all data in the training set. A hash representing a partition, histograms of classes across each client, and number of clients are stored.

CNN Architecture

Layer	MNIST configuration	Output
Convolution, ReLU, max pool	1→16; 3×3; padding 1; pool 2	16×14×14
Convolution, ReLU, max pool	16→32; 3×3; padding 1; pool 2	32×7×7
Flatten and FC, ReLU	1568→128	128
Output logits	128→10	10
Trainable parameters	206,922	827,688 bytes/message

Table 3. Corrected MNIST CNN architecture and float32 communication payload.

CIFAR 10 Materials

In CIFAR-10, we have 50,000 training and 10,000 testing RGB images of size 32 x 32 pixels. Our CNN will take in three image (RGB) channels; and has a $2048 \rightarrow 128$ fully connected layer as its first fully connected layer which results in 268,650 model parameters. We will use the same Dirichlet $\alpha = .1$ distribution for our allocations for both the local-only gossip algorithm, and the Ring gossip algorithm. MNIST and CIFAR-10 are publicly available benchmarking data sets [32] , [33].

Implementation Materials

We use Python and PyTorch as our local learning tools; we also make use of NumPy and NetworkX for data processing (e.g., numerical computations) and graph manipulation. Additionally, we use SciPy for computing student t-intervals and Matplotlib for creating figures. In order to minimize redundant computation per image, we store both the model and the input data tensors on the same machine as where they were learned. Finally, models are stored in a form which is easily loadable onto a CPU-based system for review.

3.3 Proposed Approach

Figure 1 depicts the overall process. The local-only model represents the baseline. M1 sends updates after every round of local training while M2 makes a sender choice prior to using the same receivers for an update. The third model (M3) operates independently from the perfect graph matrix. Each of these comparisons use the exact same partition and initializations with in each seed.

M1 Standard Gossip Federated Learning

Each client runs all the computations at that client's site, saves a "snapshot" of their current model parameters, and then transmits this new parameter snapshot to every other client on the network. Once each client receives the snapshot from every other client, they apply Equation (4) to update their model based upon the most recent acceptable set of data. There are no updates to any of the models until after both clients have been fully updated. In particular, the first time each pair of clients exchanges snapshots, there will be two rounds of updates. This is done so as to avoid the problem of updating one client's model before transmitting it to another client, which would alter the intended meaning of a synchronous communication round. The no-communication control does exactly the same thing, but makes no calls on the channel.

M2 Event Triggered Gossip

Beginning of Text. Every sender is comparing its post-training model to its previous transmitted snapshot, through equation (5). A triggered client will send directed model messages for each neighbor. The standard and event-triggered clients have the same receiver implementation. Reproducibility can be tested at a threshold of zero in that all parameters are hashed exactly.

M3 Spectral Gap Analysis

The Graph Matrix is examined to determine whether it has Signed Eigenvalues, Spectral Gap (SLEM), and a suitable average time in which to compute an optimal solution. The fixed matrix of [11] and [12] are used as the basis for a single variable numerical simulation where the upper-bound on the squared disagreement of all agents from their true value is computed. The training performance of neural networks is also empirically tested using local stochastic gradient descent (SGD) with missing messages; and stale arrival times that were not included within the bounds of the previously established fixed matrix proofs.

Sequence of Operation

1. Set up the seed, create benchmarks from tensors, partition clients so they are disjoint, set up a graph, load common model weights into channels, and load parameters for channels.
2. For each client's epoch of Local-SGD (the number of epochs is defined by the user), run it locally and keep the optimizer state intact after each call to Local-SGD.
3. In the case of M2, if an erasure occurs that exceeds the specified "change" threshold, allow all senders to broadcast their messages. If there was no such threshold (i.e., we're using M1), then allow all senders to broadcast their messages.
4. Simulate directed link failures (erasure) and random delays for each message that survives these failures; simulate the arrival times of all arriving messages for this round.
5. Before applying updates to each receiver, reject versions that are out-of-date and scale the available weights of neigh appropriately.
6. After completing the computation on each receiver's client-model, evaluate the model individually and collect statistics regarding the client model as follows: mean-accuracy over all examples seen during training, loss computed using the client model evaluated against its own test data, disagreement among receivers and/or between each receiver's client model and the received global model; aggregation drift between rounds of communication; and the total amount of traffic sent/received by each-client.
7. Repeat step 6 until the desired budget is reached, complete the run, save the completed run, aggregate the independent seeds used across each run and analyze the ideal graph separately.

3.4 Experimental Setup

Setting	Revised value
Independent seeds	42, 43, 44, 45, 46
Rounds / local epochs	10 / 1
Batch / learning rate / momentum	128 / 0.01 / 0.9
Core size and partitions	5 clients; IID, two-class, Dirichlet α=0.1
Scale comparison	20 and 50 clients; two-class; Ring and Grid
Loss comparison	20% Bernoulli; 20% stationary Gilbert-Elliott
Delay comparison	Maximum delay 0, 1, 2, 4 rounds
Event comparison	Dirichlet; Ring and Grid; δ=0, 0.5, 2.0

Table 4. Revised experimental protocol. The core budget is ten rounds with one local epoch. A separate matched IID control uses twenty rounds and two local epochs.

All of the client models were tested against the entire shared testing dataset. The reported accuracy values are calculated as a simple arithmetic average over all of the predictions. An ensemble model was not used to generate these predictions nor were they generated using any method that averages the predictors in some way. Training occurred on the detected NVIDIA RTX 5070 Ti laptop GPU and utilized the PyTorch package which supports automatic mixed precision for computations (i.e., it can use both single-precision floating point numbers and double-precision floating point numbers) and utilizes CUDA. The exchanged parameters in the client models remained float32 data type. Deterministic algorithms are being used for training; therefore, each training run will capture specific details regarding software version(s), device type, number of graph edges (as well as other specifics within the configuration).

Experiment Families and Evaluation Scope

Experiment family	Conditions	Seeds each	Runs
Core MNIST: three partitions × local/Ring/Grid/Random	12	5	60
Event thresholds: two topologies × two nonzero thresholds	4	5	20
Independent/burst erasures and finite delays	7	5	35
Scale: two client counts × Ring/Grid	4	5	20
Original-budget IID: local-only/Ring	2	5	10
CIFAR-10 Dirichlet: local-only/Ring	2	5	10
Total reported experiments	31	5	155

Table 5. Completed experimental design. Five additional ten-round GPU runs check M1/M2 zero-threshold equivalence and are not counted as independent comparison conditions.

The experiments compare to each other within controlled families of experiments (not as a complete product of all possible settings). The delay and burst-loss learning used the same setup with five clients in a Ring topology using Dirichlet MNIST, while zero-delay loss was evaluated independently for Grid and Random topologies. A single class is used in scale evaluation on both Ring and Grid. In addition to the 20 round IID budget used by the original experiment, the IID control also uses two epochs per client, and has batch size equal to 64. The CIFAR-10 experiment provides a further restricted comparison of the experiments.

3.5 Evaluation and Reproducibility

$$E(t) \;=\; N^{-1}\,\Sigma_{\mathrm{i}}\,||w_{\mathrm{i}}(t) \;-\; \bar{w}(t)||_{2}{}^{2} \quad (7)$$

The use of consensus error measurement represents the degree of the squared disagreement on parameters. Consensus error measurement is not a proxy for test performance — local updating (repeated) and symmetry in a neural-network model may result in an entirely new relationship between the two. Drift in the aggregation-only means refers to the magnitude of the change in the average of the network's weights, immediately after being averaged over all clients, i.e., prior to the previous local update.

All conditions were tested using five random initializations of the model parameters. Thus, each reported condition includes five independent results based upon a given set of random model parameter values. The 95% confidence interval for the difference is expressed by the equation mean ± t₀.₉₇₅,ₙ₋₁ * s / √ n. This expression captures variability due to random selection of model parameters at initialization time, data partitioning among all devices in the cluster, mini-batch assignments during the computation of gradients on each device, random sampling of graphs when necessary, and channel assignment.

This does not represent generalization to a completely different dataset or family of algorithms. Using five seeds limits the precision associated with this type of comparison. When confidence intervals overlap they are not interpreted as providing evidence of statistical significance.

Validation checks

Validation of full-budget GPUs conducts a ten-round round robin (Dirichlet) for training with all five seeds. The standard gossip as well as the zero-threshold M2 protocols produce exact client-model hash values at both the start and end of the rounds as well as exact per-round means for accuracy during the zero-loss Ring setting. Both training entry points are compared using regression analysis to the same synthetic data for loss and delays. In addition, pending arrival counts; random generation when suppressive send conditions are active; version acceptance/rejection when versions become "stale"; unequal degree messages saved; eigenvalue signatures; number of parameters; the squared error averaging bound; and average partition coverage; as well as average drift from sending to receiving in a unidirectional receive model are also tested.

Reproducible Artifacts and Data Availability

Raw revised runs for each of your experiments are located at the bottom of the results/review/runs directory. Each run has accompanying data including an overall validation record, an aggregate plot of performance over time (a summary), and legacy accounting information (a file that summarizes how many GPU hours were used). The filenames in this directory include information about which conditions were being tested, the random number seed you selected as well as the total amount of money you allocated to train a neural network using a given budget. When a trainer is finished running all of the experimental runs, it saves its output to disk completely atomically. So if there are two trainers running simultaneously, they will be able to save their outputs independently without interfering with one another. Additionally, the runners will ignore any previously completed runs when they start again. Runs that have been partially trained by the end of the day will start from scratch tomorrow morning when they are started up again.
A small portable CPU demo accompanies your final report. This is a pre-trained MNIST model and includes twenty sample images of digits to test against. There is no need to retrain the model or to download any datasets. There is also a simple command line utility included with the source code provided that can print out the experiment summary you created. Reviewing your entire experiment's results on a laptop does not require you to have the same high-end training GPU. Your final report comes complete with source code, raw results, aggregate statistics, a dependency specification, and a portable checkpoint so you can easily reproduce your results on some other machine.

4 Results

4.1 M1 Accuracy and Packet Loss Tolerance

Partition	Method	Accuracy % ± CI	Consensus error ± CI	Seeds
iid	Local only	98.03 ± 0.08	3.63 ± 0.20	5
iid	Ring	98.58 ± 0.08	0.047 ± 0.002	5
iid	Grid	98.55 ± 0.08	0.13 ± 0.01	5
iid	Random	98.56 ± 0.06	0.09 ± 0.15	5
two-class	Local only	19.953 ± 0.003	6.83 ± 1.18	5
two-class	Ring	49.27 ± 3.13	0.10 ± 0.03	5
two-class	Grid	39.22 ± 6.62	0.38 ± 0.07	5
two-class	Random	53.03 ± 16.90	0.20 ± 0.25	5
dirichlet	Local only	53.03 ± 4.55	8.18 ± 0.38	5
dirichlet	Ring	89.83 ± 1.05	0.08 ± 0.01	5
dirichlet	Grid	85.11 ± 4.40	0.24 ± 0.14	5
dirichlet	Random	88.25 ± 9.45	0.19 ± 0.37	5

Table 6. MNIST results at N=5, ten rounds, one local epoch, and no loss or delay. Intervals are 95% across five seeds.

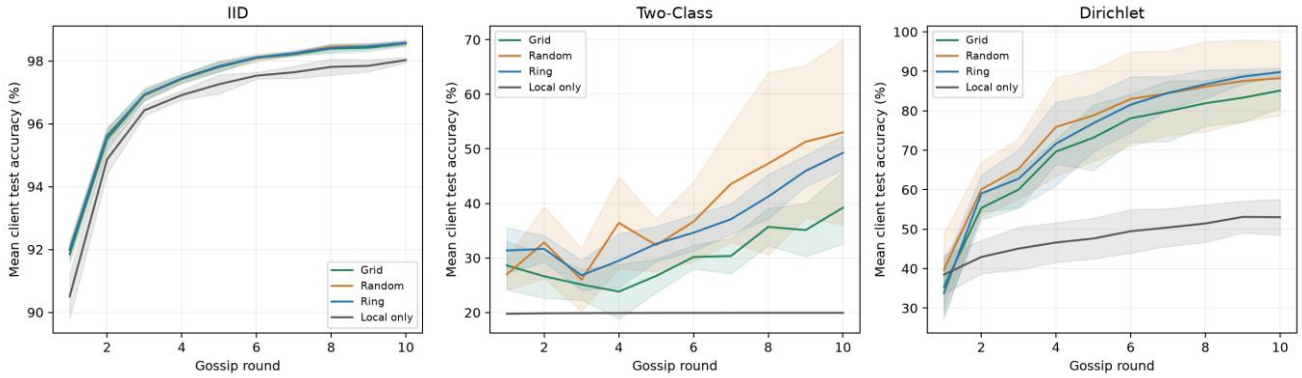


Figure 2. Mean client accuracy versus round, with 95% confidence bands. Conditions share data and initialization within each seed.

IID clients already have access to every class of data, while two-class clients may never be able to determine the full label space from only their own samples. It's evidence of the appropriate advantage in terms of communication that each partition has. The learning curves indicate an intermediary behavior where a fully-saturated accuracy value could conceal. These results do not prove robustness to impairment(s) which are not represented on this table.

4.1.1 Paired Gains over Local Only

Partition	Topology	Accuracy gain over local only in points	Pairs
iid	Ring	0.55 ± 0.03	5
iid	Grid	0.53 ± 0.03	5
iid	Random	0.54 ± 0.10	5
two-class	Ring	29.31 ± 3.13	5
two-class	Grid	19.27 ± 6.63	5
two-class	Random	33.07 ± 16.89	5
dirichlet	Ring	36.80 ± 4.63	5
dirichlet	Grid	32.08 ± 4.25	5
dirichlet	Random	35.22 ± 10.08	5

Table 7. Within-seed accuracy differences. Each condition has five matched pairs.

4.2 M2 Event Triggered Communication

Topology	δ	Accuracy % ± CI	Message savings % ± CI	Broadcast suppression % ± CI
Ring	0.0	89.83 ± 1.05	0.00 ± 0.00	0.00 ± 0.00
Ring	0.5	76.26 ± 5.94	25.20 ± 6.71	25.20 ± 6.71
Ring	2.0	62.37 ± 3.51	79.60 ± 4.44	79.60 ± 4.44
Grid	0.0	85.11 ± 4.40	0.00 ± 0.00	0.00 ± 0.00
Grid	0.5	78.56 ± 6.86	25.00 ± 13.26	27.20 ± 11.46
Grid	2.0	60.12 ± 4.37	82.20 ± 2.39	82.00 ± 1.76

Table 8. Event-triggered Dirichlet MNIST. Actual message savings measure payload reduction; broadcast suppression uses a different denominator.

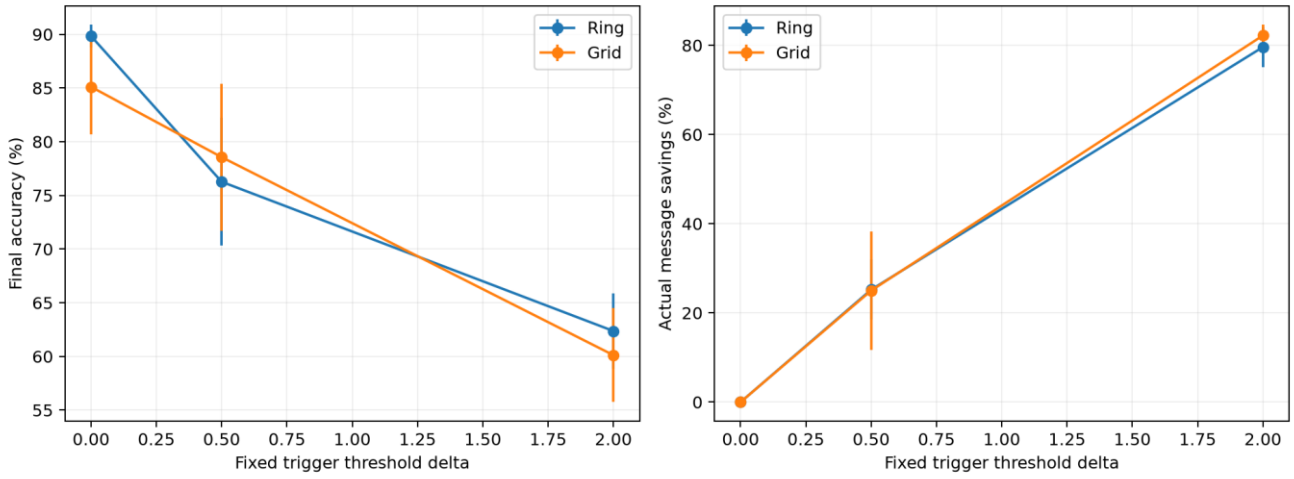


Figure 3. Accuracy and actual message savings versus fixed threshold, with 95% seed intervals.

RING mean message savings at Δ = 2.0 is 79.6%, but mean accuracy changed from 89.83% to 62.37%. The values represent the mean of the fixed-threshold method (not an assumption that there would be no reduction in accuracy through compression).

Silent broadcasts may slow down the propagation of information about classes. Therefore, traffic savings does not indicate negligible loss of accuracy.

An absolute parameter-distance threshold is unique to the architecture, partition, and training-budget used; it cannot be assumed to be applicable across architectures without testing.

4.3 Packet Loss Delay and Burst Erasures

Topology	Channel	Max delay	Accuracy % ± CI	Consensus error ± CI	Mean drift
Grid	bernoulli	0	82.10 ± 4.48	0.34 ± 0.17	0.0975
Random	bernoulli	0	85.94 ± 10.11	0.28 ± 0.47	0.0960
Ring	bernoulli	0	86.39 ± 1.63	0.15 ± 0.04	0.1338
Ring	gilbert-elliott	0	85.46 ± 4.20	0.26 ± 0.20	0.1708
Ring	bernoulli	1	78.34 ± 4.47	0.32 ± 0.14	0.2392
Ring	bernoulli	2	71.35 ± 8.33	0.52 ± 0.20	0.2887
Ring	bernoulli	4	69.22 ± 9.88	0.80 ± 0.45	0.3192

Table 9. Impaired-channel results on Dirichlet MNIST. The delay and burst comparisons use Ring; independent zero-delay loss also compares Grid and Random. Drift averages the aggregation-only norm across rounds and seeds. Bernoulli and burst channels match nominal long-run erasure probability, not realized losses.

The zero-delay Bernoulli runs erase 21.0% of attempted messages on average, compared with 22.6% for Gilbert-Elliott. Both models have nominal long-run loss 20%; the finite-run realizations need not coincide.

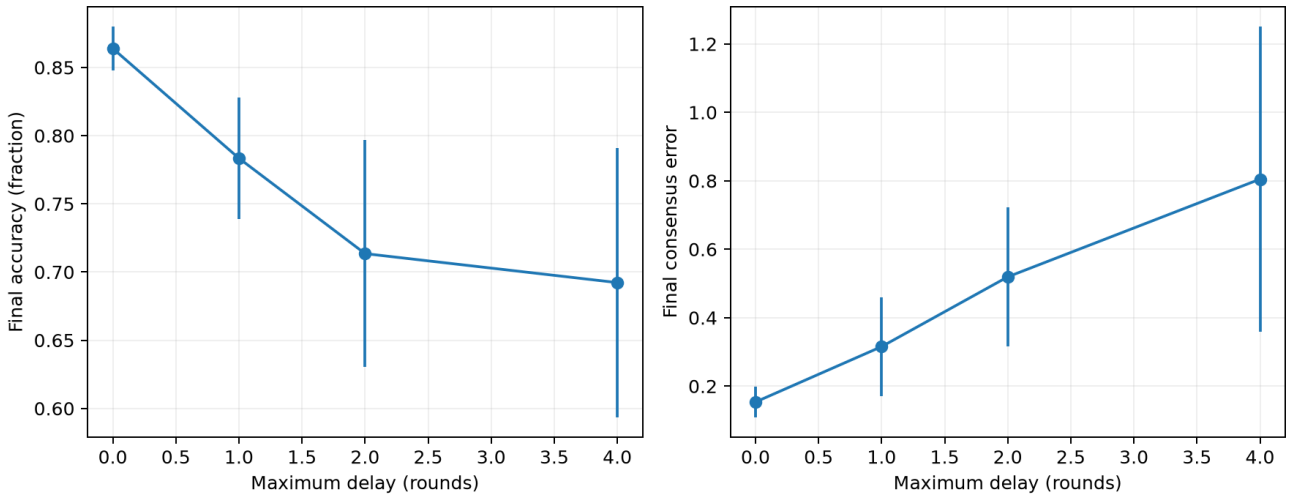


Figure 4. Delay sensitivity with independent erasure probability 0.2. Individual surviving messages can still have zero delay.

These runs test the delay model using fixed amounts for both the local training budget, as well as the gossip round budget. There are three possible explanations if you observe a difference (newest version rule), which includes missing packets, models being outdated, or how much different stations intermingle.

The Newest Version Rule does not allow an older packet to replace a newer acceptable model. Thus, even though you have observed a positive result on your delay, it does not necessarily follow that all arbitrary latency will be benign.

The Burst Model begins in stationarity, however, there are still only ten rounds of observations for a wireless system whose mean time spent in a "bad state" is five rounds. Thus, in addition to accuracy, realized pending/dropped count values need to also be examined.

4.4 Scale and CIFAR 10 Extension

Clients	Topology	Accuracy % ± CI	Consensus error ± CI	Attempts
5	Ring	49.27 ± 3.13	0.10 ± 0.03	100
5	Grid	39.22 ± 6.62	0.38 ± 0.07	100
20	Ring	37.53 ± 4.80	0.51 ± 0.19	400
20	Grid	24.57 ± 1.20	0.61 ± 0.06	620
50	Ring	36.92 ± 3.11	0.43 ± 0.09	1000
50	Grid	55.17 ± 2.80	0.35 ± 0.07	1700

Table 10. Actual neural-network training at N=5, 20, and 50, using two-class MNIST. Total dataset size is fixed.

The ring average has an accuracy of 49.27% when the number of nodes (N) equals five and 36.92% when the number of nodes equals fifty using this fixed round/total amount of data method for the scale comparison. These uncertainty intervals and traffic counts can be seen in the scale comparison graph above.
As stated earlier, this is a model-training comparison for client models, rather than simply a larger eigenvalue calculation. Each client receives two labels , however these two labels have to be split among a greater number of clients with increasing values of N. While total data are fixed, the number of example datasets that are provided to each client and the size of minibatches decrease. For all values of N, both Ring and Grid have identical training budget partitions.

CIFAR-10 method	Accuracy % ± CI	Consensus error ± CI	Seeds
Local only	28.29 ± 2.84	19.18 ± 1.52	5
Ring gossip	38.36 ± 3.15	0.44 ± 0.04	5

Table 11. CIFAR-10 with five clients, Dirichlet α=0.1, and the same ten-round budget.

CIFAR-10 includes 50,000 training and 10,000 testing RGB images of size 32 × 32.
Means for each color-channel = [0.4914 0.4822 0.4465], Standard Deviations = [0.2470 0.2435 0.2616].
The Two-Convolutions Model contains three color-channels as inputs to an initial fully-connected layer with an output-size of 128 units from 2048 units.
Thus total number of model parameters = 268,650.
This document presents a study comparing local-only gossip and Ring gossip that applies only to a specific two convolutional-layer CIFAR-10 protocol.
There was no data-augmentation applied or CNN architecture search done.
In this experiment using only the Ring Gossip model achieved mean accuracy of >10% over the local-only gossip model.
As noted above, the table indicates seed-to-seed variation in both average accuracies.
We have preserved the paired-difference in the seed-to-seed variations in the accompanying summary json file.
Therefore, the results we present below can be taken as a controlled-comparison on CIFAR-10 vs. other CIFAR-10 benchmarks.
We report absolute accuracy and any comparative accuracy gain under these same conditions.
Test results were produced without us retroactively adjusting the budget to achieve some desired outcome.

4.5 M3 Spectral Gap and Convergence Analysis

Spectral Gap

The ideal spectral gap $\gamma = 1 - \rho$ uses the second-largest eigenvalue modulus ρ. The signed spectrum and the magnitude used for SLEM are retained separately.

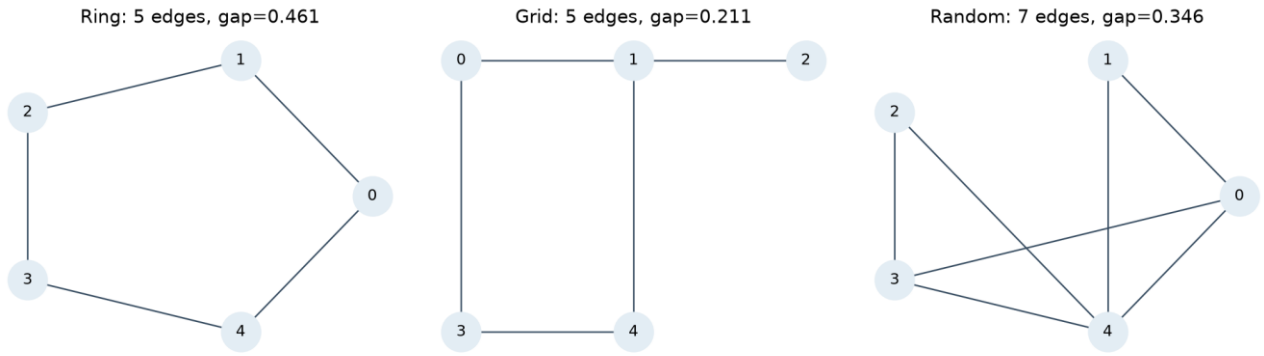


Figure 5. Actual five-node graphs at seed 42. The trimmed Grid has degree sequence 2, 3, 1, 2, 2, including a pendant node.

Topology	Edges	Gap γ	SLEM ρ	T(.01)	T(.001)
Ring	5	0.460655	0.539345	4	6
Grid	5	0.211325	0.788675	10	15
Random	7	0.346447	0.653553	6	9

Table 12. Ideal five-node spectra. T is a sufficient integer number of rounds for a relative squared-disagreement target.

Eigenvalue Spectra

Topology	λ_1	λ_2	λ_3	λ_4	λ_5
Ring	1.000000	0.539345	0.539345	-0.206011	-0.206011
Grid	1.000000	0.788675	0.416667	-0.250000	0.211325
Random	1.000000	0.653553	0.300000	-0.053553	0.000000

Table 13. Signed eigenvalues sorted by descending magnitude. SLEM uses magnitude; the full spectrum preserves signs.

Convergence Bound

For fixed symmetric doubly stochastic W, let $J = 11^T/N$ and $\rho = \|W - J\|_2$. Disagreement under pure averaging is $(W-J)^t X(0)$. Applying the spectral-norm bound to squared Frobenius disagreement gives [11], [12]:

$$E(t) \leq E(0)\,\rho^{2t}, \quad \gamma = 1 - \rho \quad (8)$$

Mixing Time

$$T(\varepsilon) = ceiling\{\, log(1/\varepsilon) \,/\, [2\, log(1/\rho)] \,\} \quad (9)$$

Equation (9) targets $E(t)/E(0) \leq \varepsilon$. An unsquared-norm target would remove the factor of two in the denominator. The table therefore states the relative squared-disagreement convention and reports sufficient integer round counts.

4.6 Spectral Validation and Comparison with M1

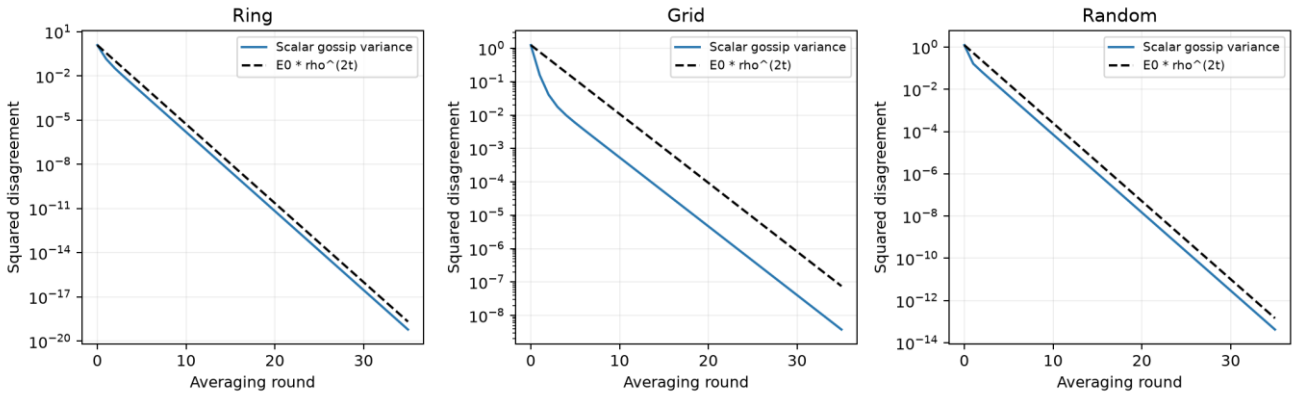


Figure 6. Scalar averaging variance and the squared-error bound on identical graph instances. This checks ideal averaging, not SGD convergence.

A scalar trajectory will lie below the bound due to roundoff error in practice; it doesn't have to lie on the bound since the projected component of the initial vector onto the slowest mode can be very small. At some point near machine precision, we should not interpret what appears to be a convergence slope as indicating that our physical problem has converged.

Cross Comparison with M1

Local SGD periodically introduces disagreements in addition to changing loss, suppressing events, and introducing stale data into the update operators over time. This larger set of influences cannot be covered by a fixed matrix proof; therefore, the larger "ideal" spectral gap will not ensure better model accuracy for models that are learned using this algorithm nor smaller final learning disagreements.

Previous saved results provide counterexamples to an unconditional ranking. Under a loss level of less than twenty percent (and assuming that all other parameters were identical), Random had an average consensus error value of approximately 0.161 whereas Ring had an average consensus error of 0.301, even though the ideal Ring spectral gap was greater. As such, the current analysis considers the previously described empirical measures of performance as comparative measures rather than confirmation of an unconditional measure of the ideal spectral prediction.

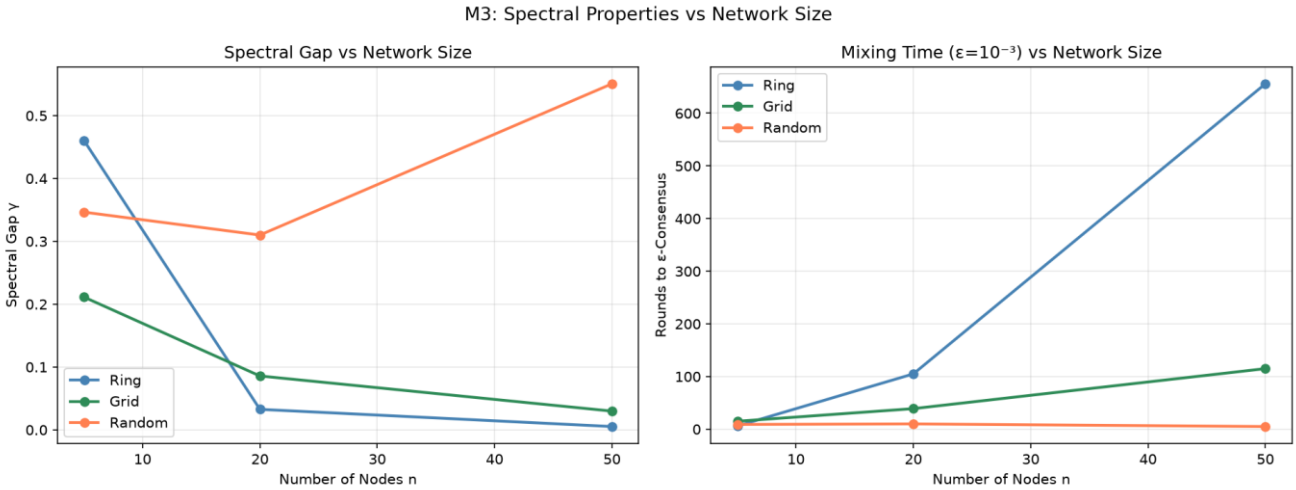


Figure 7. Ideal gap and squared-error mixing bound at N=5, 20, and 50. Fixed p=0.5 Random graphs become denser as N increases.

5 Discussion

5.1 Why Ring and Grid Performance Depends on Network Size

The five-node Ring has an ideal gap of .461, while the trimmed five-node Grid has an ideal gap of .211. These values can explain why there was a difference in how well the networks mixed rather than show that Rings are always better than Grids. The larger network example also changed the number of pieces of data that each client had to work with since the total number of pieces of data did not change.

For two-class MNIST, we see that at five nodes, the average accuracy of all clients is 49.27% for Ring and 39.22% for Grid. However, as you increase the number of clients from 5 to 50, we see that the average accuracy increases to 55.17% for Grid and decreases to 36.92% for Ring. We observe this pattern based on the number of clients in the network and our complete training protocol. Therefore, given the same number of nodes, we would expect both algorithms to utilize the same data allocation.

5.2 Robustness to Packet Loss and Relation to Theory

To be clear, the results of our testing under the Dirichlet MNIST testbed show that as we increase both the amount of data lost (from zero to twenty percent) and the maximum number of rounds of delayed reception, we see corresponding decreases in the average ring accuracy -- from 89.83% in no-loss scenarios through 86.39% when there was an average of 20% loss, to 69.22%, where the maximum number of rounds of delayed receipt was four rounds with 20% loss. The statistics provide a characterization of how different budgets and policies affect each other. We also report confidence intervals for these tests along with the actual volume of traffic associated with those tests.

We are unable to use similar performance over a single communication channel as evidence that it would be acceptable to have arbitrary loss or delay.

5.3 Event Triggered Communication Analysis

When $\delta = 2.0$ at Ring, it was possible to save as much as 79.6 % of all the messages sent (directed) in order to save money; however, we had lost almost one-third of the accuracy we originally achieved (accuracy went down from 89.83 % to 62.37 %). In addition to the above issue, there was also a cost vs. accuracy trade-off with respect to traffic. Any threshold that is set using an absolute number will be dependent on both the model used and the user's budget for this particular application. As such, there are two ways to achieve compression or adaptive peer selection for a specific design option [9][10]; however, these cannot be accomplished solely by increasing this threshold.
On an irregular graph, a silent high degree client eliminates more direct messages than a silent low degree client. Therefore, in our simulations, we count broadcast suppression separately from the amount of traffic saved. For example, when looking at the original Grid counts data, there were 8 suppressed broadcasts out of 100 potential broadcast opportunities. However, only 188 attempts to send messages to neighbors were made instead of 200. In this run, we experienced 6 percent less payload traffic due to 8 percent fewer broadcasts.

5.4 Limitations

Five seeds make up the initial estimate of uncertainty. Using a same benchmark test suite does not take place of a test suite to evaluate new populations. No validation-based hyperparameter search was run. There is a uniform weighting of the client objectives. This is different from a pooled sample-weighted objective for unequal numbers of data. Momentum in each client's local network is maintained through out the process (i.e., after mixing). The length of the core horizon is small. Larger network comparisons will be based upon the total number of data points available as opposed to the number of data points that can be obtained by each client. The channel discards all information regarding the actual messages sent from models and also removes physical packetization, retrying, measured bandwidth, power consumption, and real wall clock latency. Integer round delay is not a completely asynchronous execution model. Honest static clients were assumed. Differential privacy, secure aggregation, Byzantine resistance, mobile vehicle topologies dynamically changing, and average preserving lossy aggregation have been excluded from this implementation [28] [30] [31]. Only local only and Ring learning are compared with the CIFAR-10 extension and there is no evidence that these results indicate superiority over the other referenced algorithms.

5.5 Communication Benefit under the Original Training Budget

Local only mean accuracy at 20 rounds, 2 local epochs, batch size 64 was 98.52 ± .07%. This is compared to 99.17 ± .09% for ring gossip. There was a five-seed paired gain of 0.65 ± .02 percentage points when using communication. Therefore, while we expected that communication would make a contribution in this IID setting, it was smaller than in the heterogeneous short-budget comparisons.

5.6 Findings in Relation to the Research Gap

The Local-Only control determines if communication helps given a specific budget (the Non-IID test demonstrates the importance of using common classes); the Channel tests demonstrate how "stale" or "missing" info affects accuracy; Event-Triggered tests show how the amount of traffic translates into accuracy; and together, these studies provide answers to the Evaluation Questions of Sec. 2.4 without providing better bounds on all possible wireless scenarios than those already surveyed.

Some additional areas that have been identified as future work are: More Heterogeneous Wireless Networks; Changing Network Topologies; Resource Constraints in Wireless networks; Privacy Mechanisms for Aggregation; Alternative Aggregation Policies [29], [25], [26], [27]; As such, a more complete benchmarking effort would be to re-implement some of the algorithms with similar budgets.

6 Conclusion

Communication can be valuable for clients when data is distributed among sites. For example, in the two class (0 vs. 1) MNIST classification problem, use of ring gossip increases the average client accuracy by about 30 percentage points relative to client-only training (from 19.95% to 49.27%). In the iid (identically independently distributed) case, however, all clients achieve nearly perfect accuracy (>98%) as if they had been able to train together. This comparison illustrates how heterogeneity in the data affects our interpretation of how useful the communication mechanism is.
In this paper we evaluate several well-known methods of decentralized learning with controls that match the behavior of these methods without their use of inter-client communication, with each site having different data, with five seeds used to generate randomness in the selection of initial parameters for the models at each site, and with accurate counting of messages transmitted. We separate out the performance achieved by a model trained with one method from the desired properties of the matrix that mixes the examples used to train it. While conclusions are drawn based upon specific partitions, budget sizes, types of graphs and models of channels used in this evaluation, no generalizations are made regarding whether there is a universally effective packet loss mitigation strategy or whether there is a generally preferred type of graph structure for the ring topology over other topologies such as grid. Empirical tradeoffs in terms of accuracy versus traffic are determined using directed message counts and actual payload bytes. Since delay may occur during communication due to processing time at either end of each link, delay is also evaluated using version aware receivers and a finite horizon. All of the results presented in this paper have been saved so that they could be reproduced by someone else using a script and a specification of dependencies. Additionally, a simple example demonstrates the ability to reproduce results on a single laptop by saving a run and then checking point in order to execute it again.

6.1 Future Work

Future assessment should measure both of these selected unreliable link and communication efficient algorithms with the same data, training, and bytes budget. Additional extensions to consider are average preserving aggregation in a directed loss context; time varying graphs; using validation to determine the best thresholds for each algorithm; extending the harder task length during training; as well as fairness to clients. Privacy protection and vehicular mobility will require specific methods that provide privacy protection independent of the fact that they can be executed on a server free architecture [28][20][31]

7 Declarations

Funding: There is no mention of funding for this project. Conflict(s) of Interest: The authors have made no declarations regarding conflicts of interest in the context of this project. Availability of Data: MNIST and CIFAR-10 were previously released as open-source benchmarks [32][33]. Source code and results: This project also includes source code with dependency descriptions; raw results; statistics; figures; validation test cases; and a portable checkpoint that demonstrates the functionality of the evaluation. The provided artifacts include the implementation used to evaluate it and all evidence collected to support those evaluations.

8 References

[1] McMahan, B., Moore, E., Ramage, D., Hampson, S., Aguera y Arcas, B. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, PMLR 54, 1273-1282. https://proceedings.mlr.press/v54/mcmahan17a.html

[2] Lian, X., Zhang, C., Zhang, H., Hsieh, C.-J., Zhang, W., Liu, J. (2017). Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent. Advances in Neural Information Processing Systems 30. https://arxiv.org/abs/1705.09056

[3] Sun, T., Li, D., Wang, B. (2023). Decentralized Federated Averaging. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(4), 4289-4301. https://doi.org/10.1109/tpami.2022.3196503

[4] Savazzi, S., Nicoli, M., Rampa, V. (2020). Federated Learning With Cooperating Devices: A Consensus Approach for Massive IoT Networks. IEEE Internet of Things Journal, 7(5), 4641-4654. https://doi.org/10.1109/jiot.2020.2964162

[5] Lalitha, A., Kilinc, O.C., Javidi, T., Koushanfar, F. (2019). Peer-to-peer Federated Learning on Graphs. arXiv preprint arXiv:1901.11173. https://arxiv.org/abs/1901.11173

[6] Ye, H., Liang, L., Li, G.Y. (2022). Decentralized Federated Learning With Unreliable Communications. IEEE Journal of Selected Topics in Signal Processing, 16(3), 487-500. https://doi.org/10.1109/jstsp.2022.3152445

[7] Jiang, Z., Yu, G., Cai, Y., Jiang, Y. (2022). Decentralized Edge Learning via Unreliable Device-to-Device Communications. IEEE Transactions on Wireless Communications, 21(11), 9041-9055. https://doi.org/10.1109/twc.2022.3172147

[8] Li, W., Lv, T., Ni, W., Zhao, J., Hossain, E., Poor, H.V. (2024). Decentralized Federated Learning Over Imperfect Communication Channels. IEEE Transactions on Communications, 72(11), 6973-6991. https://doi.org/10.1109/tcomm.2024.3407208

[9] Koloskova, A., Stich, S.U., Jaggi, M. (2019). Decentralized Stochastic Optimization and Gossip Algorithms with Compressed Communication. Proceedings of the 36th International Conference on Machine Learning, PMLR 97, 3478-3487. https://proceedings.mlr.press/v97/koloskova19a.html

[10] Tang, Z., Shi, S., Li, B., Chu, X. (2023). GossipFL: A Decentralized Federated Learning Framework With Sparsified and Adaptive Communication. IEEE Transactions on Parallel and Distributed Systems, 34(3), 909-922. https://doi.org/10.1109/tpds.2022.3230938

[11] Xiao, L., Boyd, S. (2004). Fast linear iterations for distributed averaging. Systems & Control Letters, 53(1), 65-78. https://doi.org/10.1016/j.sysconle.2004.02.022

[12] Boyd, S., Ghosh, A., Prabhakar, B., Shah, D. (2006). Randomized gossip algorithms. IEEE Transactions on Information Theory, 52(6), 2508-2530. https://doi.org/10.1109/tit.2006.874516

[13] Hegedűs, I., Danner, G., Jelasity, M. (2019). Gossip Learning as a Decentralized Alternative to Federated Learning. Lecture Notes in Computer Science, 74-90. https://doi.org/10.1007/978-3-030-22496-7_5

[14] Gao, H., Thai, M.T., Wu, J. (2023). When Decentralized Optimization Meets Federated Learning. IEEE Network, 37(5), 233-239. https://doi.org/10.1109/mnet.132.2200530

[15] Liu, W., Chen, L., Zhang, W. (2022). Decentralized Federated Learning: Balancing Communication and Computing Costs. IEEE Transactions on Signal and Information Processing over Networks, 8, 131-143. https://doi.org/10.1109/tsipn.2022.3151242

[16] Xing, H., Simeone, O., Bi, S. (2021). Federated Learning Over Wireless Device-to-Device Networks: Algorithms and Convergence Analysis. IEEE Journal on Selected Areas in Communications, 39(12), 3723-3741. https://doi.org/10.1109/jsac.2021.3118400

[17] Chen, M., Gündüz, D., Huang, K., Saad, W., Bennis, M., Feljan, A.V., Poor, H.V. (2021). Distributed Learning in Wireless Networks: Recent Progress and Future Challenges. IEEE Journal on Selected Areas in Communications, 39(12), 3579-3605. https://doi.org/10.1109/jsac.2021.3118346

[18] Yang, H.H., Liu, Z., Quek, T.Q.S., Poor, H.V. (2020). Scheduling Policies for Federated Learning in Wireless Networks. IEEE Transactions on Communications, 68(1), 317-333. https://doi.org/10.1109/tcomm.2019.2944169

[19] Salehi, M., Hossain, E. (2021). Federated Learning in Unreliable and Resource-Constrained Cellular Wireless Networks. IEEE Transactions on Communications, 69(8), 5136-5151. https://doi.org/10.1109/tcomm.2021.3081746

[20] Li, Z., Zhou, Y., Wu, D., Tang, T., Wang, R. (2022). Fairness-Aware Federated Learning With Unreliable Links in Resource-Constrained Internet of Things. IEEE Internet of Things Journal, 9(18), 17359-17371. https://doi.org/10.1109/jiot.2022.3156046

[21] Xu, C., Liu, S., Yang, Z., Huang, Y., Wong, K.-K. (2021). Learning Rate Optimization for Federated Learning Exploiting Over-the-Air Computation. IEEE Journal on Selected Areas in Communications, 39(12), 3742-3756. https://doi.org/10.1109/jsac.2021.3118402

[22] Amiri, M.M., Gündüz, D., Kulkarni, S.R., Poor, H.V. (2022). Convergence of Federated Learning Over a Noisy Downlink. IEEE Transactions on Wireless Communications, 21(3), 1422-1437. https://doi.org/10.1109/twc.2021.3103874

[23] Wei, X., Shen, C. (2022). Federated Learning Over Noisy Channels: Convergence Analysis and Design Examples. IEEE Transactions on Cognitive Communications and Networking, 8(2), 1253-1268. https://doi.org/10.1109/tccn.2022.3140788

[24] Nedić, A., Olshevsky, A., Rabbat, M.G. (2018). Network Topology and Communication-Computation Tradeoffs in Decentralized Optimization. Proceedings of the IEEE, 106(5), 953-976. https://doi.org/10.1109/jproc.2018.2817461

[25] Martínez Beltrán, E.T., Quiles Pérez, M., Sánchez Sánchez, P.M., López Bernal, S., Bovet, G., Gil Pérez, M., Martínez Pérez, G., Huertas Celdrán, A. (2023). Decentralized Federated Learning: Fundamentals, State of the Art, Frameworks, Trends, and Challenges. IEEE Communications Surveys & Tutorials, 25(4), 2983-3013. https://doi.org/10.1109/comst.2023.3315746

[26] Yuan, L., Wang, Z., Sun, L., Yu, P.S., Brinton, C.G. (2024). Decentralized Federated Learning: A Survey and Perspective. IEEE Internet of Things Journal, 11(21), 34617-34638. https://doi.org/10.1109/jiot.2024.3407584

[27] Chung, W.-C., Lo, C.-A., Lin, Y.-H., Chen, Z.-H., Hung, C.-L. (2026). Decentralized Federated Learning with Non-IID Data: Challenges, Trends, and Future Opportunities. ACM Computing Surveys, 58(8), 1-41. https://doi.org/10.1145/3785657

[28] Chen, S., Yu, D., Zou, Y., Yu, J., Cheng, X. (2022). Decentralized Wireless Federated Learning With Differential Privacy. IEEE Transactions on Industrial Informatics, 18(9), 6273-6282. https://doi.org/10.1109/tii.2022.3145010

[29] Imteaj, A., Thakker, U., Wang, S., Li, J., Amini, M.H. (2022). A Survey on Federated Learning for Resource-Constrained IoT Devices. IEEE Internet of Things Journal, 9(1), 1-24. https://doi.org/10.1109/jiot.2021.3095077

[30] Nguyen, M.N.H., Pandey, S.R., Dang, T.N., Huh, E.-N., Tran, N.H., Saad, W., Hong, C.S. (2023). Self-Organizing Democratized Learning: Toward Large-Scale Distributed Learning Systems. IEEE Transactions on Neural Networks and Learning Systems, 34(12), 10698-10710. https://doi.org/10.1109/tnnls.2022.3170872

[31] Chen, D., Deng, T., Jia, J., Feng, S., Yuan, D. (2025). Mobility-aware decentralized federated learning with joint optimization of local iteration and leader selection for vehicular networks. Computer Networks, 263, 111232. https://doi.org/10.1016/j.comnet.2025.111232

[32] LeCun, Y., Cortes, C., Burges, C. J. C. (n.d.). The MNIST database of handwritten digits. Dataset: 60,000 training and 10,000 test images. https://www.tensorflow.org/datasets/catalog/mnist

[33] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. University of Toronto technical report; CIFAR-10 dataset. https://www.cs.toronto.edu/~kriz/cifar.html